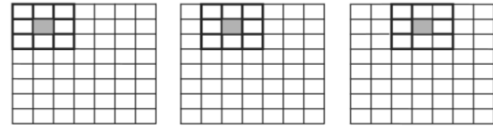


Filtro de Suavização

Patrícia Dockhorn Costa, pdcosta@inf.ufes.br



Em processamento de imagens, um filtro de suavização pode ser usado para reduzir o ruído de uma imagem. A ideia geral é que cada pixel possa ser substituído pela média dos valores dos vizinhos imediatos (com uma determinada janela). Nesta prova, considere que a entrada configure uma matriz de valores de pixels (em escala cinza). Por exemplo, o arquivo de entrada (entrada.txt) a seguir:

```
4 5
10 10 200 200 200
10 10 200 200 200
10 10 200 200 200
10 10 200 200 200
```

A primeira linha contém 2 números (4 e 5) que representam as dimensões (linhas e colunas) da “matriz” de pixels. As linhas seguintes contêm os valores dos pixels, inteiros entre 0 e 255.

Para gerar as saídas esperadas, o programa deve:

1. Carregar os dados de entrada em uma **matriz dinâmica**;
2. Criar uma “matriz suavizada” com as mesmas dimensões da matriz original. Na matriz suavizada, cada elemento é a média dos valores de seus vizinhos imediatos, incluindo ele mesmo (veja o algoritmo ao final do enunciado). Nas bordas da imagem (ou matriz), os elementos permanecem inalterados;
3. Salvar a matriz original e a matriz de suavização em um arquivo de saída (saída.txt);
4. Liberar toda a memória alocada! Use valgrind...

Para a matriz de entrada acima, espera-se a seguinte saída (saida.txt):

```
10 10 200 200 200
10 10 200 200 200
10 10 200 200 200
10 10 200 200 200
```

A matriz com filtro de suavização eh:

```
10 10 200 200 200
10 73 136 200 200
10 73 136 200 200
10 10 200 200 200
```

O algoritmo de suavização é o seguinte:

```
Para cada pixel (i, j):  
    se (elemento de borda da matriz)  
        m_suavizada[i][j] = original[i][j]  
    senão:  
        soma = 0  
        contador = 0  
        para x de i-1 até i+1:  
            para y de j-1 até j+1:  
                soma += original[x][y]  
                contador++  
        m_suavizada [i][j] = soma / contador
```

Regras muito importantes:

- Implementar um TAD Matriz, como discutimos em sala de aula. Ou seja, espero, ao menos, 3 arquivos na resposta (2 para o TAD e um para o cliente – main);
- Usar somente **alocação dinâmica** para vetores e matrizes;
- Usar tipo opaco estruturado (estilo: `typedef struct x Tx`), como discutimos em sala de aula;
- Manipulação de arquivo de entrada somente na *main* (não manipular arquivo de entrada no TAD);
- Pode-se imprimir a matriz no arquivo de saída na função de “imprimir” do TAD;
- Liberar toda a memória alocada! Usar o `valgrind`...